

Lab Assignment 3

CMPE 146 Section 1

Professor Bhawandeep Singh Harsh

Andrew Bertch

Robert Trinh

Table of Contents

Exercise 1 – Semaphore	3
Exercise 2 – Synchronized Control.....	5
Exercise 3 – Multiple Tasks to Compute in Parallel.....	7
Appendix A – MCU Source Code.....	10
semaphore.c	10
multiple.c.....	17
parallel.c.....	28

Exercise 1 – Semaphore

In this exercise, we create two different tasks to blink the red LED on the LaunchPad. We have one task for the control and the other for the timing. The functions to complete these tasks are included below.

```
void *timing(void *arg0)
{
    while(1)
    {
        usleep(500000);
        sem_post(&some_semaphore);
    }
}
```

This is the timing function, and it is a very simple infinite while loop and a sleep. By calling `sem_post` the semaphore is unlocked and its value increased to tell the control function to change the LED state.

```
void *control(void *arg1)
{
    DL_GPIO_enablePower(GPIOA);

    DL_GPIO_initDigitalOutput(IOMUX_PINCM1); // PA0, LED1

    DL_GPIO_initDigitalInput(IOMUX_PINCM1); // PA0, LED1

    DL_GPIO_enableOutput(GPIOA, DL_GPIO_PIN_0); //PA0, LED1

    DL_GPIO_clearPins(GPIOA, DL_GPIO_PIN_0);

    int state = 0;
```

```

while(1)
{
    sem_wait(&some_semaphore);

    if(!state)
    {
        DL_GPIO_setPins(GPIOA, DL_GPIO_PIN_0);
    }
    else
    {
        DL_GPIO_clearPins(GPIOA, DL_GPIO_PIN_0);
    }

    state = !state;
}
}

```

This is the control function, and it sets up the LED and GPIO pins to work and then waits for the semaphore to be unlocked or > 0. When this occurs, the state variable is checked for whether or not to turn the LED on or off and then state is inverted.

By adding an additional semaphore, we can have the timing task wait for the control task to become active before it starts its oscillation. This extra semaphore ensures that the timer doesn't build up a large value on the first semaphore. A large value on the first semaphore would cause the control task to rapidly blink the LED as it works through its commands and the sem_wait's.

Exercise 2 – Synchronized Control

In this exercise, we create multiple tasks to control different LEDs independently with the same task function. This allows us to not have to duplicate code when using multiple devices of the same type. Making use of the program from the previous exercise we can create a very simple synchronized control loop for all the devices. Some modifications were made to the timing function. These changes are shown below.

```
void *timing(void *arg0)
{
    usleep(rand() % 200000);
    while (1)
    {
        int hz = (rand() % 10) + 1;
        int period = 1000000 / hz;

        for (int i = 0; i < 3; i++)
            sem_post(&some_semaphore);

        usleep(10000);

        for (int i = 0; i < 3; i++)
            sem_post(&some_semaphore);

        usleep(period - 10000);
    }
}
```

The changes are to account for the 3 different devices we are now controlling and for the random frequency generation. Since we are now controlling 3 devices with the semaphore, each time we want them to change states we have to post to the semaphore

3 times. We also were tasked with having the blink frequency change with time between 1-10 Hz.

```
void *control(void *arg1)
{
    usleep(rand() % 200000);
    threadArgs *data = (threadArgs*) arg1;

    DL_GPIO_enablePower(data->gpio);

    DL_GPIO_initDigitalOutput(data->ioMuxPin);

    DL_GPIO_initDigitalInput(data->ioMuxPin);

    DL_GPIO_enableOutput(data->gpio, data->dlGpioPin);

    DL_GPIO_clearPins(data->gpio, data->dlGpioPin);

    int state = 0;

    while(1)
    {
        sem_wait(&some_semaphore);
        if(!state)
        {
            DL_GPIO_setPins(data->gpio, data->dlGpioPin);
        }
        else
```

```

    {
        DL_GPIO_clearPins(data->gpio, data->dlGpioPin);
    }

    state = !state;

    sched_yield();

}
}

```

The control function also changed a bit in order to support the struct passed to it that is necessary to hold all of the data for controlling each LED. Other than that, the while loop and control logic is the same as the previous control function in Exercise 1.

Exercise 3 – Multiple Tasks to Compute in Parallel

In this exercise, we were tasked with writing a program that could break a process into parts and then use a semaphore to synchronize all the tasks at the end for completion of the whole process. To do this, we would calculate large factorials, in this case 165!. Synchronization was completed by having two different task functions. The first task function was the master function. This function has some extra code from the normal factorial function. It has a for loop that is waiting for the control semaphore to count up to the number of normal threads to complete their sections. The normal factorial function threads calculate the factorial for their part of the data and then post to the semaphore and increasing the value of the semaphore's count. The outputs of this whole program are included below showing the proper value being calculated for 165!

CORTEX_MoP: task: 0 start: 1 end: 55

CORTEX_MoP: task: 1 start: 56 end: 110

CORTEX_MoP: task: 2 start: 111 end: 165

CORTEX_MoP: task: 1 progress: 6.553814e+08

CORTEX_MoP: task: 0 progress: 3.628800e+06

CORTEX_MoP: task: 2 progress: 4.211882e+20

CORTEX_MoP: task: 1 progress: 9.434618e+26

CORTEX_MoP: task: 0 progress: 2.432902e+18

CORTEX_MoP: task: 2 progress: 4.071698e+41

CORTEX_MoP: task: 1 progress: 5.636987e+45

CORTEX_MoP: task: 0 progress: 2.652529e+32

CORTEX_MoP: task: 2 progress: 8.476027e+62

CORTEX_MoP: task: 1 progress: 1.170186e+65

CORTEX_MoP: task: 0 progress: 8.159153e+47

CORTEX_MoP: task: 2 progress: 3.597293e+84

CORTEX_MoP: task: 1 progress: 7.350603e+84

CORTEX_MoP: task: 0 progress: 3.041409e+64

CORTEX_MoP: task: 2 progress: 2.968511e+106

CORTEX_MoP: task: 1 progress: 1.250941e+105

CORTEX_MoP: task 1 done

CORTEX_MoP: task 2 done

CORTEX_MoP: final value: 5.423911e+295

Appendix A – MCU Source Code

semaphore.c

```
/* Standard includes */
#include <stdio.h>

/* POSIX header files */
#include <pthread.h>

/* RTOS header files */
#include <FreeRTOS.h>
#include <task.h>

#include <unistd.h>
#include <semaphore.h>

/* TI includes for driver configuration */
#include "ti_msp_dl_config.h"

sem_t some_semaphore;

void *timing(void *arg0)
{
    while(1)
    {
        usleep(500000);
        sem_post(&some_semaphore);
    }
}
```

```

    }
}

void *control(void *arg1)
{
    DL_GPIO_enablePower(GPIOA);

    DL_GPIO_initDigitalOutput(IOMUX_PINCM1); // PA0, LED1

    DL_GPIO_initDigitalInput(IOMUX_PINCM1); // PA0, LED1

    DL_GPIO_enableOutput(GPIOA, DL_GPIO_PIN_0); //PA0, LED1

    DL_GPIO_clearPins(GPIOA, DL_GPIO_PIN_0);

    int state = 0;

    while(1)
    {
        sem_wait(&some_semaphore);

        if(!state)
        {
            DL_GPIO_setPins(GPIOA, DL_GPIO_PIN_0);
        }
        else
        {

```

```

        DL_GPIO_clearPins(GPIOA, DL_GPIO_PIN_0);
    }

    state = !state;

}

}

/* Stack size in bytes */
#define THREADSTACKSIZE 1024

/* Set up the hardware ready to run this demo */
static void prvSetupHardware(void);

int main(void)
{
    pthread_t threadTiming, threadControl;
    pthread_attr_t attrs;
    struct sched_param priParam;
    int retcTiming, retcControl;

    /* Initialize the system locks */
#ifdef __ICCARM__
    __iar_Initlocks();
#endif

```

```

/* Prepare the hardware to run this demo. */
prvSetupHardware();

/* Initialize the attributes structure with default values */
pthread_attr_init(&attrs);

/* Set priority, detach state, and stack size attributes */
priParam.sched_priority = 1;
retcTiming                = pthread_attr_setschedparam(&attrs,
&priParam);
retcTiming |= pthread_attr_setdetachstate(&attrs,
PTHREAD_CREATE_DETACHED);
retcTiming |= pthread_attr_setstacksize(&attrs, THREADSTACKSIZE);
retcControl                =
pthread_attr_setschedparam(&attrs, &priParam);
retcControl |= pthread_attr_setdetachstate(&attrs,
PTHREAD_CREATE_DETACHED);
retcControl |= pthread_attr_setstacksize(&attrs, THREADSTACKSIZE);
if (retcTiming != 0) {
    /* failed to set attributes */
    while (1) {
    }
}
if (retcControl != 0) {
    /* failed to set attributes */
    while (1) {
    }
}

```

```

    sem_init(&some_semaphore, 0, 0);
    retcTiming = pthread_create(&threadTiming, &attrs, timing, NULL);
    retcControl = pthread_create(&threadControl, &attrs, control,
NULL);
    if (retcTiming != 0) {
        /* pthread_create() failed */
        while (1) {
            }
        }
    if (retcControl != 0) {
        /* pthread_create() failed */
        while (1) {
            }
        }

    /* Start the FreeRTOS scheduler */
    vTaskStartScheduler();
    sem_destroy(&some_semaphore);

    return (0);
}

static void prvSetupHardware(void)
{
    SYSCFG_DL_init();
}

```

```

// #if (configSUPPORT_STATIC_ALLOCATION == 1)
// /*
// * ===== vApplicationGetIdleTaskMemory =====
// * When static allocation is enabled, the app must provide this
// callback
// * function for use by the Idle task.
// */
// void vApplicationGetIdleTaskMemory(StaticTask_t
**ppxIdleTaskTCBBuffer,
//      StackType_t **ppxIdleTaskStackBuffer, uint32_t
*pulIdleTaskStackSize)
// {
//      static StaticTask_t xIdleTaskTCB;
//      static StackType_t
uxIdleTaskStack[configIDLE_TASK_STACK_DEPTH];

//      *ppxIdleTaskTCBBuffer = &xIdleTaskTCB;
//      *ppxIdleTaskStackBuffer = uxIdleTaskStack;
//      *pulIdleTaskStackSize = configIDLE_TASK_STACK_DEPTH;
// }

// #if (configUSE_TIMERS == 1)
// /*
// * ===== vApplicationGetTimerTaskMemory =====
// * When static allocation is enabled, and timers are used, the app
// must provide
// * this callback function for use by the Timer Service task.

```

```

// */
// void vApplicationGetTimerTaskMemory(StaticTask_t
**ppxTimerTaskTCBBuffer,
//      StackType_t **ppxTimerTaskStackBuffer, uint32_t
*pulTimerTaskStackSize)
// {
//      static StaticTask_t xTimerTaskTCB;
//      static StackType_t
uxTimerTaskStack[configTIMER_TASK_STACK_DEPTH];

//      *ppxTimerTaskTCBBuffer    = &xTimerTaskTCB;
//      *ppxTimerTaskStackBuffer = uxTimerTaskStack;
//      *pulTimerTaskStackSize    = configTIMER_TASK_STACK_DEPTH;
// }
// #endif

// #endif

#if (configCHECK_FOR_STACK_OVERFLOW)
/*
    * ===== vApplicationStackOverflowHook =====
    * When stack overflow checking is enabled the application must
provide a
    * stack overflow hook function. This default hook function is
declared as
    * weak, and will be used by default, unless the application
specifically
    * provides its own hook function.
    */

```

```

#if defined(__IAR_SYSTEMS_ICC__)
__weak void vApplicationStackOverflowHook(
    TaskHandle_t pxTask, char *pcTaskName)
#elif (defined(__TI_COMPILER_VERSION__))
#pragma WEAK(vApplicationStackOverflowHook)
void vApplicationStackOverflowHook(TaskHandle_t pxTask, char
*pcTaskName)
#elif (defined(__GNUC__) || defined(__ti_version__))
void __attribute__((weak))
vApplicationStackOverflowHook(TaskHandle_t pxTask, char *pcTaskName)
#endif
{
    /* default to spin upon stack overflow */
    while (1) {
        }
    }
#endif

```

multiple.c

```

/* Standard includes */
#include <stdio.h>
#include <stdlib.h>

/* POSIX header files */
#include <pthread.h>

/* RTOS header files */
#include <FreeRTOS.h>

```

```

#include <task.h>

#include <unistd.h>
#include <semaphore.h>

/* TI includes for driver configuration */
#include "ti_msp_dl_config.h"

sem_t some_semaphore;

typedef struct {
    GPIO_Regs * gpio;
    int ioMuxPin;
    int dlGpioPin;
} threadArgs;

void *timing(void *arg0)
{

    usleep(rand() % 200000);
    while (1)
    {
        int hz = (rand() % 10) + 1;
        int period = 1000000 / hz;

        for (int i = 0; i < 3; i++)
            sem_post(&some_semaphore);
    }
}

```

```

        usleep(10000);

        for (int i = 0; i < 3; i++)
            sem_post(&some_semaphore);

        usleep(period - 10000);
    }
}

void *control(void *arg1)
{
    usleep(rand() % 200000);
    threadArgs *data = (threadArgs*) arg1;

    DL_GPIO_enablePower(data->gpio);

    DL_GPIO_initDigitalOutput(data->ioMuxPin);

    DL_GPIO_initDigitalInput(data->ioMuxPin);

    DL_GPIO_enableOutput(data->gpio, data->dlGpioPin);

    DL_GPIO_clearPins(data->gpio, data->dlGpioPin);

    int state = 0;

```

```

while(1)
{

    sem_wait(&some_semaphore);

    if(!state)
    {
        DL_GPIO_setPins(data->gpio, data->dlGpioPin);
    }
    else
    {
        DL_GPIO_clearPins(data->gpio, data->dlGpioPin);
    }

    state = !state;

    sched_yield();

}

}

/* Stack size in bytes */
#define THREADSTACKSIZE 1024

```

```

/* Set up the hardware ready to run this demo */
static void prvSetupHardware(void);

int main(void)
{
    srand(12345);

    pthread_t threadRED, threadGREEN, threadBLUE, threadTiming;
    pthread_attr_t attrs;
    struct sched_param priParam;
    int retcRED, retcGREEN, retcBLUE, retcTiming;

    /* Initialize the system locks */
#ifdef __ICCARM__
    __iar_Initlocks();
#endif

    /* Prepare the hardware to run this demo. */
    prvSetupHardware();

    /* Initialize the attributes structure with default values */
    pthread_attr_init(&attrs);

    /* Set priority, detach state, and stack size attributes */
    priParam.sched_priority = 1;

    retcTiming = pthread_attr_setschedparam(&attrs, &priParam);

```

```

    retcTiming |= pthread_attr_setdetachstate(&attrs,
PTHREAD_CREATE_DETACHED);

    retcTiming |= pthread_attr_setstacksize(&attrs, THREADSTACKSIZE);


    retcRED  = pthread_attr_setschedparam(&attrs, &priParam);
    retcRED  |= pthread_attr_setdetachstate(&attrs,
PTHREAD_CREATE_DETACHED);
    retcRED  |= pthread_attr_setstacksize(&attrs, THREADSTACKSIZE);


    retcGREEN = pthread_attr_setschedparam(&attrs, &priParam);
    retcGREEN |= pthread_attr_setdetachstate(&attrs,
PTHREAD_CREATE_DETACHED);
    retcGREEN |= pthread_attr_setstacksize(&attrs, THREADSTACKSIZE);


    retcBLUE  = pthread_attr_setschedparam(&attrs, &priParam);
    retcBLUE  |= pthread_attr_setdetachstate(&attrs,
PTHREAD_CREATE_DETACHED);
    retcBLUE  |= pthread_attr_setstacksize(&attrs, THREADSTACKSIZE);


    if (retcTiming != 0) {
        /* failed to set attributes */
        while (1) {
        }
    }

    if (retcRED != 0) {
        /* failed to set attributes */
        while (1) {
        }
    }

```

```
}  
if (retcGREEN != 0) {  
    /* failed to set attributes */  
    while (1) {  
    }  
}  
if (retcBLUE != 0) {  
    /* failed to set attributes */  
    while (1) {  
    }  
}
```

```
threadArgs redArgs;  
redArgs.gpio = GPIOA;  
redArgs.ioMuxPin = IOMUX_PINCM1;  
redArgs.dlGpioPin = DL_GPIO_PIN_0;
```

```
threadArgs greenArgs;  
greenArgs.gpio = GPIOB;  
greenArgs.ioMuxPin = IOMUX_PINCM58;  
greenArgs.dlGpioPin = DL_GPIO_PIN_27;
```

```
threadArgs blueArgs;  
blueArgs.gpio = GPIOB;  
blueArgs.ioMuxPin = IOMUX_PINCM50;  
blueArgs.dlGpioPin = DL_GPIO_PIN_22;
```

```
sem_init(&some_semaphore, 0, 0);  
retcTiming = pthread_create(&threadTiming, &attrs, timing, NULL);  
retcRED = pthread_create(&threadRED, &attrs, control, &redArgs);  
retcGREEN = pthread_create(&threadGREEN, &attrs, control,  
&greenArgs);  
retcBLUE = pthread_create(&threadBLUE, &attrs, control,  
&blueArgs);
```

```
if (retcTiming != 0) {  
    /* pthread_create() failed */  
    while (1) {  
    }  
}  
if (retcRED != 0) {  
    /* pthread_create() failed */  
    while (1) {  
    }  
}  
if (retcGREEN != 0) {  
    /* pthread_create() failed */  
    while (1) {  
    }  
}  
if (retcBLUE != 0) {  
    /* pthread_create() failed */
```

```

        while (1) {
            }
        }

    /* Start the FreeRTOS scheduler */
    vTaskStartScheduler();
    sem_destroy(&some_semaphore);

    return (0);
}

static void prvSetupHardware(void)
{
    SYSCFG_DL_init();
}

// #if (configSUPPORT_STATIC_ALLOCATION == 1)
// /*
//  * ===== vApplicationGetIdleTaskMemory =====
//  * When static allocation is enabled, the app must provide this
//  callback
//  * function for use by the Idle task.
//  */
// void vApplicationGetIdleTaskMemory(StaticTask_t
// **ppxIdleTaskTCBBuffer,
// //      StackType_t **ppxIdleTaskStackBuffer, uint32_t
// *pulIdleTaskStackSize)

```

```

// {
//     static StaticTask_t xIdleTaskTCB;
//     static StackType_t
uxIdleTaskStack[configIDLE_TASK_STACK_DEPTH];

//     *ppxIdleTaskTCBBuffer    = &xIdleTaskTCB;
//     *ppxIdleTaskStackBuffer = uxIdleTaskStack;
//     *pulIdleTaskStackSize    = configIDLE_TASK_STACK_DEPTH;
// }

// #if (configUSE_TIMERS == 1)
// /*
// * ===== vApplicationGetTimerTaskMemory =====
// * When static allocation is enabled, and timers are used, the app
must provide
// * this callback function for use by the Timer Service task.
// */
// void vApplicationGetTimerTaskMemory(StaticTask_t
**ppxTimerTaskTCBBuffer,
//     StackType_t **ppxTimerTaskStackBuffer, uint32_t
*pulTimerTaskStackSize)
// {
//     static StaticTask_t xTimerTaskTCB;
//     static StackType_t
uxTimerTaskStack[configTIMER_TASK_STACK_DEPTH];

//     *ppxTimerTaskTCBBuffer    = &xTimerTaskTCB;
//     *ppxTimerTaskStackBuffer = uxTimerTaskStack;

```

```

//      *pulTimerTaskStackSize    = configTIMER_TASK_STACK_DEPTH;
// }
// #endif

// #endif

#if (configCHECK_FOR_STACK_OVERFLOW)
/*
 * ===== vApplicationStackOverflowHook =====
 * When stack overflow checking is enabled the application must
provide a
 * stack overflow hook function. This default hook function is
declared as
 * weak, and will be used by default, unless the application
specifically
 * provides its own hook function.
 */
#if defined(__IAR_SYSTEMS_ICC__)
__weak void vApplicationStackOverflowHook(
    TaskHandle_t pxTask, char *pcTaskName)
#elif (defined(__TI_COMPILER_VERSION__))
#pragma WEAK(vApplicationStackOverflowHook)
void vApplicationStackOverflowHook(TaskHandle_t pxTask, char
*pcTaskName)
#elif (defined(__GNUC__) || defined(__ti_version__))
void __attribute__((weak))
vApplicationStackOverflowHook(TaskHandle_t pxTask, char *pcTaskName)
#endif

```

```
{  
    /* default to spin upon stack overflow */  
    while (1) {  
    }  
}  
#endif
```

parallel.c

```
/* Standard includes */  
#include <stdio.h>  
#include <stdlib.h>  
  
/* POSIX header files */  
#include <pthread.h>  
  
/* RTOS header files */  
#include <FreeRTOS.h>  
#include <task.h>  
  
#include <unistd.h>  
#include <semaphore.h>  
  
/* TI includes for driver configuration */  
#include "ti_msp_dl_config.h"  
  
sem_t some_semaphore;  
  
#define FACT_NUM 165
```

```
#define NTASKS 3
```

```
struct app_task_params {  
    int index;  
    double partial_result;  
    int start;  
    int end;  
};
```

```
struct app_task_params task_params_list[NTASKS];
```

```
void *master(void *arg0)
```

```
{
```

```
    struct app_task_params *data = arg0;
```

```
    printf("task: %d\tstart: %d\tend: %d\n", data[0].index,  
data[0].start, data[0].end);
```

```
    data[0].partial_result = data[0].start;
```

```
    for (int i = data[0].start + 1; i <= data[0].end; i++) {
```

```
        data[0].partial_result *= i;
```

```
        usleep(1000);
```

```
        if (i % 10 == 0) {
```

```
            printf("task: %d\tprogress: %.6e\n", data[0].index,  
data[0].partial_result);
```

```
        }
```

```
    }
```

```
    int sval = -1;
```

```

    for (int i = 1; i < NTASKS; i++) {
        sem_wait(&some_semaphore);
    }

    for (int i = 1; i < NTASKS; i++) {
        data[0].partial_result *= data[i].partial_result;
    }
    printf("final value: %e\n", data[0].partial_result);

    sched_yield();
    return 0;
}

void *factorial(void *arg1)
{
    struct app_task_params *data = arg1;

    printf("task: %d\tstart: %d\tend: %d\n", data->index, data->start,
data->end);

    data->partial_result = data->start;
    for (int i = data->start + 1; i <= data->end; i++) {
        data->partial_result *= i;
        usleep(1000);
        if (i % 10 == 0) {
            printf("task: %d\tprogress: %.6e\n", data->index, data-
>partial_result);
        }
    }
}

```

```

    }

    printf("task %d done\n", data->index);
    sem_post(&some_semaphore);

    sched_yield();
    return 0;
}

/* Stack size in bytes */
#define THREADSTACKSIZE 2048

/* Set up the hardware ready to run this demo */
static void prvSetupHardware(void);

int main(void)
{
    pthread_t threads[NTASKS];
    pthread_attr_t attrs;
    struct sched_param priParam;
    int retc;

    /* Initialize the system locks */
#ifdef __ICCARM__
    __iar_Initlocks();
#endif

    /* Prepare the hardware to run this demo. */

```

```

prvSetupHardware();

/* Initialize the attributes structure with default values */
pthread_attr_init(&attrs);
pthread_attr_setdetachstate(&attrs, PTHREAD_CREATE_DETACHED);

/* Set priority, detach state, and stack size attributes */
priParam.sched_priority = 1;
retc                      = pthread_attr_setschedparam(&attrs,
&priParam);
retc |= pthread_attr_setdetachstate(&attrs,
PTHREAD_CREATE_DETACHED);
retc |= pthread_attr_setstacksize(&attrs, THREADSTACKSIZE);
if (retc != 0) {
    /* failed to set attributes */
    while (1) {
        }
    }

for (int i = 0; i < NTASKS; i++) {
    task_params_list[i].index = i;
    task_params_list[i].start = 1 + (FACT_NUM / NTASKS * i);
    task_params_list[i].end = (FACT_NUM / NTASKS) + (FACT_NUM /
NTASKS * i);
}

sem_init(&some_semaphore, 0, 0);

```

```

    retc = pthread_create(&threads[0], &attrs, master,
&task_params_list[0]);
    if (retc != 0) {
        /* pthread_create() failed */
        printf("thread 0 failed\n");
        while (1) {
            return -1;
        }
    }
    for (int i = 1; i < NTASKS; i++) {
        retc = pthread_create(&threads[i], &attrs, factorial,
&task_params_list[i]);
        if (retc != 0) {
            /* pthread_create() failed */
            printf("thread %d failed\t %d", i, retc);
            while (1) {
                return -1;
            }
        }
    }

    /* Start the FreeRTOS scheduler */
    vTaskStartScheduler();
    sem_destroy(&some_semaphore);

    return (0);
}

```

```

static void prvSetupHardware(void)
{
    SYSCFG_DL_init();
}

// #if (configSUPPORT_STATIC_ALLOCATION == 1)
// /*
//  * ===== vApplicationGetIdleTaskMemory =====
//  * When static allocation is enabled, the app must provide this
//  callback
//  * function for use by the Idle task.
//  */
// void vApplicationGetIdleTaskMemory(StaticTask_t
// **ppxIdleTaskTCBBuffer,
// StackType_t **ppxIdleTaskStackBuffer, uint32_t
// *pulIdleTaskStackSize)
// {
//     static StaticTask_t xIdleTaskTCB;
//     static StackType_t
// uxIdleTaskStack[configIDLE_TASK_STACK_DEPTH];

//     *ppxIdleTaskTCBBuffer    = &xIdleTaskTCB;
//     *ppxIdleTaskStackBuffer = uxIdleTaskStack;
//     *pulIdleTaskStackSize    = configIDLE_TASK_STACK_DEPTH;
// }

// #if (configUSE_TIMERS == 1)

```

```

// /*
// * ===== vApplicationGetTimerTaskMemory =====
// * When static allocation is enabled, and timers are used, the app
must provide
// * this callback function for use by the Timer Service task.
// */
// void vApplicationGetTimerTaskMemory(StaticTask_t
**ppxTimerTaskTCBBuffer,
//      StackType_t **ppxTimerTaskStackBuffer, uint32_t
*pulTimerTaskStackSize)
// {
//      static StaticTask_t xTimerTaskTCB;
//      static StackType_t
uxTimerTaskStack[configTIMER_TASK_STACK_DEPTH];

//      *ppxTimerTaskTCBBuffer    = &xTimerTaskTCB;
//      *ppxTimerTaskStackBuffer = uxTimerTaskStack;
//      *pulTimerTaskStackSize    = configTIMER_TASK_STACK_DEPTH;
// }
// #endif

// #endif

#if (configCHECK_FOR_STACK_OVERFLOW)
/*
* ===== vApplicationStackOverflowHook =====
* When stack overflow checking is enabled the application must
provide a

```

```

    * stack overflow hook function. This default hook function is
    declared as

    * weak, and will be used by default, unless the application
    specifically

    * provides its own hook function.

    */
#ifdef __IAR_SYSTEMS_ICC__
__weak void vApplicationStackOverflowHook(
    TaskHandle_t pxTask, char *pcTaskName)
#elif (defined(__TI_COMPILER_VERSION__))
#pragma WEAK(vApplicationStackOverflowHook)
void vApplicationStackOverflowHook(TaskHandle_t pxTask, char
*pcTaskName)
#elif (defined(__GNUC__) || defined(__ti_version__))
void __attribute__((weak))
vApplicationStackOverflowHook(TaskHandle_t pxTask, char *pcTaskName)
#endif

{
    /* default to spin upon stack overflow */
    while (1) {
        }
    }
#endif

```